SENA

National Learning Service (SENA)

Software Analysis and Development – 3145556

PROJECT

EduAirControl

General Project Document

Overview, scope, architecture, organization and status of the project

MEMBERS

María de los Ángeles Olaya García

Jhon Jairo Cortez Vargas

INSTRUCTORS

All program instructors

Version 1.0 · October 9, 2026 · 2025-2026

Table of contents

1. General project information . . . 3

2. Introduction . . . 3

3. Problem statement . . . 4

4. Purpose and justification . . . 4

4.1 Purpose . . . 4
4.2 Justification . . . 4

5. Objectives . . . 4

5.1 General objective . . . 4
5.2 Specific objectives . . . 4

6. Scope . . . 5

6.1 Capabilities included in the MVP . . . 5
6.2 Out of scope for this version . . . 5
6.3 Constraints and assumptions . . . 6

7. Product description . . . 6

7.1 Users and roles . . . 6
7.2 Monitored variables . . . 6
7.3 Main user journeys . . . 7

8. Architecture and technology . . . 7

8.1 Backend services . . . 7
8.2 Technology stack . . . 8
8.3 IoT hardware . . . 9

9. Team and work organization . . . 9

9.1 Team . . . 9
9.2 Methodology . . . 9

10. Current status and planning . . . 10

10.1 Requirements status . . . 10
10.2 Project milestones . . . 10
10.3 Roadmap . . . 10

11. Main risks . . . 11

12. Project deliverables . . . 11

13. Glossary . . . 12

14. References . . . 12

1. General project information

This document gives a complete, high-level view of **EduAirControl**: the problem it solves, who it serves, what it includes and excludes, how it is built, who builds it, how the work is managed and what its current status is. It is the entry point to the rest of the project deliverables; each topic is covered in depth in the document listed in section 12.

Field	Detail
Project name	EduAirControl
Type of solution	IoT-based school environmental monitoring system with a web application, a mobile application and a microservices backend
Training program	Technologist in Software Analysis and Development – 3145556
Institution	National Learning Service – SENA
Team members	María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas
Instructors	All program instructors
Period	2025-2026
Product status	MVP under development (unreleased version)
Baseline documentation	SRS v1.1 (IEEE 830), updated on 2026-10-08 against the code and the engineering documentation
Documentation repository	github.com/code-sena/ea-control-docs
Version of this document	1.0 – October 9, 2026

2. Introduction

Well-being in school environments has become a key factor for quality education. Environmental conditions inside the classroom directly affect the concentration, academic performance and health of students and teachers. In this context, technological solutions are needed to monitor and improve these spaces.

EduAirControl is a school environmental monitoring system based on IoT (Internet of Things) technology. It records the temperature, carbon dioxide (CO_2), humidity and noise level of educational environments and presents them in a web application and a mobile application, together with severity-based alerts, historical analysis and recommendations to improve the measured conditions.

Besides optimizing the physical conditions of the classroom, the project seeks to foster a culture of environmental and technological awareness in the educational community, in line with the principles of sustainability, health and digital transformation.

3. Problem statement

In many educational institutions, environmental conditions (temperature, CO_2, humidity and noise) are not monitored continuously. These variables are invisible, yet they affect the concentration, learning and health of students and teachers. Without real-time information it is not possible to make timely decisions, such as ventilating a room when CO_2 rises, and the measurements that do exist are scattered and lack a history that shows changes over time.

> **Research question.** How can continuous monitoring of environmental variables such as temperature, carbon dioxide, humidity and noise level in educational institutions improve the concentration, learning and health of students and teachers?

4. Purpose and justification

4.1 Purpose

To develop and implement a school environmental monitoring system that uses IoT technology to record the classroom's environmental variables in real time, improve the physical conditions of the educational setting and support, with data, decisions that strengthen the quality of learning.

4.2 Justification

According to the World Health Organization guidelines (2021), poor ventilation in enclosed spaces such as classrooms lets CO_2 build up, causing drowsiness, fatigue and reduced attention. Satish et al. (2012) found negative effects on decision-making at CO_2 concentrations above 1,000 ppm. The EPA (2020) also reports that exposure to high temperatures can reduce student performance and that excess noise increases the likelihood of errors in schoolwork.

In Colombia, most public educational institutions lack systems to monitor these variables. EduAirControl responds with an accessible solution: it collects and analyzes environmental data in real time, generates alerts and statistics useful for pedagogical and administrative decisions, and promotes technology adoption, aligning with the guidelines of the Ministry of National Education and the Sustainable Development Goals of the 2030 Agenda.

5. Objectives

5.1 General objective

To develop an IoT-based school environmental monitoring system that records and displays temperature, carbon dioxide (CO_2), humidity and noise level in real time, in order to improve learning conditions and the well-being of students and teachers in educational environments.

5.2 Specific objectives

- Design a software system that integrates sensors for real-time monitoring of temperature, CO_2, humidity and noise in the educational environments of one or several institutions.
- Develop an intuitive interface that lets administrators and teachers view environmental data and receive alerts when conditions fall outside the optimal ranges.

- Create an analysis module that processes the measurement history by day, week, month and year to identify patterns and support decision-making.
- Integrate a severity-based alert and notification system that informs those responsible about anomalies and enables immediate corrective action.
- Guarantee secure access, isolated by institution and campus, through standard OAuth2/OpenID Connect authentication and role-based access control.

6. Scope

6.1 Capabilities included in the MVP

#	Capability	Description	Responsible service
1	Authentication and registration	Sign-up, login (email and Google/Facebook), password recovery and role-based access control.	ms-security
2	Institutions and campuses	Creation and management of institutions (tenants) and campuses; data isolation per institution.	ms-security, ms-classroom-management
3	Educational environments	Creation, query, editing and deletion of environments with type, location and capacity.	ms-classroom-management
4	Sensor management	Registration, installation/removal and configuration of sensors and the variables they measure.	ms-sensor-management
5	Environmental monitoring	Reception and storage of measurements; current status of temperature, humidity, CO_2 and noise.	ms-environment-monitoring
6	Dashboard and historical analysis	Current and historical visualization by day, week, month and year.	ms-environment-monitoring, frontend
7	Alerts and notifications	Threshold-based alerts (INFO, WARNING, CRITICAL) and per-user read status.	ms-environment-monitoring, ms-user-experience
8	Favorites, ratings and preferences	Favorite environments, 1-to-5 rating with comment, language, theme and accessibility.	ms-user-experience
9	Client applications	Web application (React) and mobile application (Expo/React Native) with BLE configuration of ESP32 devices.	frontend

6.2 Out of scope for this version

- Advanced environmental prediction.
- Automatic equipment control (ventilation, air conditioning).
- Integration with external environmental control systems.
- Development of the sensor hardware (supplied by a provider or the institution).

6.3 Constraints and assumptions

- The application requires internet access and IoT devices must be powered on and connected to a Wi-Fi network.
- Base technology: React, Expo/React Native, Spring Boot, PostgreSQL 15, Redis, RabbitMQ and Docker.
- Work is organized in one-week sprints.
- The budget for sensors and microcontrollers is limited, which constrains the quantity and quality of the equipment.
- Each user belongs to a single institution and a single campus and cannot see data from other institutions.
- Stable Wi-Fi and adequate power supply are assumed in the spaces, and that the chosen sensors have acceptable accuracy for basic monitoring.

7. Product description

7.1 Users and roles

Role	Profile	What they do in the system
SUPER_ADMIN	Platform technical team	Creates and manages institutions; manages users and roles across the whole platform; handles errors and supports administrators.
ADMIN	Person in charge of an institution, with office-software skills	Manages the environments, sensors, devices and thresholds of their institution.
USER	Teachers, administrators and students	Views environments and variables, filters, marks favorites, rates environments, reviews alerts and sets preferences.

7.2 Monitored variables

Variables and their ranges are defined once (ADR-014) and are used identically by the backend, the web and the mobile app. Normal, warning and critical thresholds are configured per environment type (classroom, laboratory and library).

Variable	Code	Unit	Valid range	Classroom example: normal
Temperature	TEMP	°C	-10 to 50	20 – 26
Humidity	HUM	%	0 to 100	40 – 60
Carbon dioxide	CO2	ppm	0 to 5000	0 – 800
Noise level	NOISE	dB	0 to 140	0 – 45

7.3 Main user journeys

- **Authorized consultation:** the user signs up with their institution code or logs in, picks an environment, sees its real-time metrics and queries the history by day, week, month or year.
- **Administrative setup:** the administrator creates the environment, registers the sensor, installs it in the environment, links the variables it measures and defines the thresholds.
- **From measurement to decision:** the sensor sends a measurement; the system validates it, stores it, compares it with the thresholds and raises an alert if needed; the user sees it on the dashboard and acts (for example, ventilates the room).

8. Architecture and technology

EduAirControl follows a microservices architecture with Spring Boot behind an API Gateway that acts as a BFF (Backend for Frontend). Each service has its own database schema (ADR-003), is organized with hexagonal architecture (ADR-009) and communicates with the others through REST or RabbitMQ events.

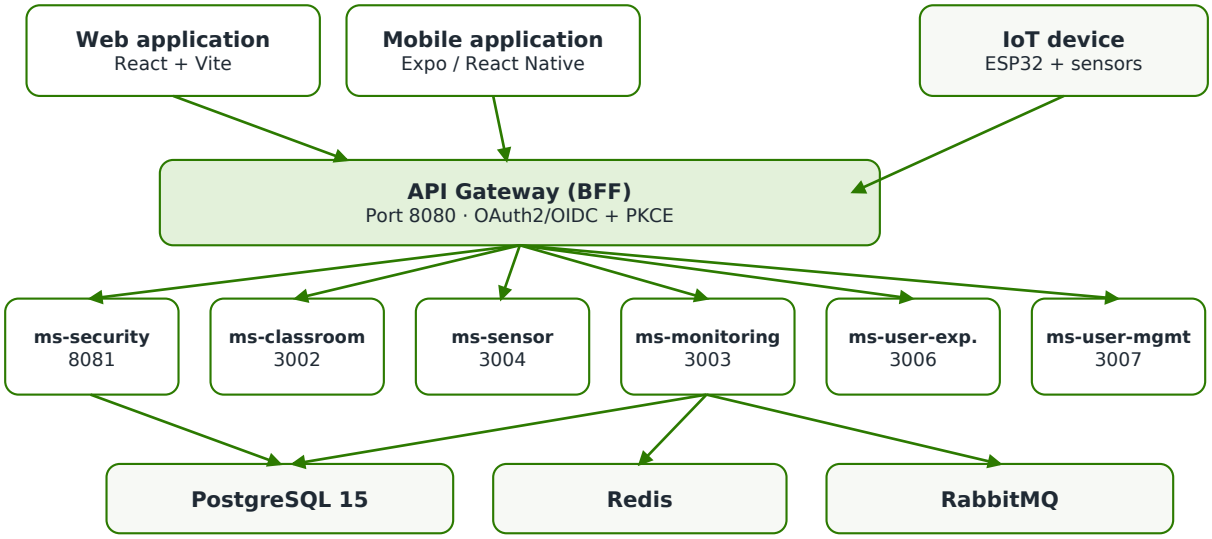


Figure 1. Overview of EduAirControl components. Full detail is in 05_Diseno_Software.

8.1 Backend services

Service	Responsibility	Port	Status
api-gateway	Single entry point, authentication, routing and rate limiting (BFF).	8080	Active
ms-security	Sign-up, login, OAuth2/OIDC, tokens, roles and institutions.	8081	In development
ms-classroom-management	Educational environments, environment types and campuses.	3002	In development
ms-environment-monitoring	Measurements, historical analysis, thresholds and alerts.	3003	Active
ms-sensor-management	Sensors, variables and installations.	3004	In development
ms-user-experience	Favorites, ratings, preferences, searches and notification status.	3006	Active
ms-user-management	User information.	3007	In development
ms-alert	Retired: its functions moved to ms-environment-monitoring (ADR-015).	—	Deprecated

8.2 Technology stack

Layer	Technology	Role
Web frontend	React 19, Vite	Responsive web interface with light/dark mode and color-blindness themes
Mobile app	Expo 54, React Native 0.81	Android and iOS app with BLE device configuration
Backend	Java 17, Spring Boot	Domain services and APIs under hexagonal architecture
API entry	Spring Cloud Gateway	Routing, authentication, rate limiting and httpOnly cookie session
Authentication	Spring Authorization Server, JWT RS256	OAuth2/OIDC with Authorization Code + PKCE; Google and Facebook
Data	PostgreSQL 15, Liquibase	Relational persistence and schema version control
Support	Redis, RabbitMQ	Sessions and revoked tokens; events between services
Contracts	OpenAPI	One contract per service in 07-api/contracts/openapi
DevOps	Docker, Jenkins, GitHub	Containers, continuous integration and source control
Observability	Prometheus, Grafana, OpenTelemetry	Metrics, dashboards and traces (defined in ADR-012)

8.3 IoT hardware

The measurement prototype is based on an **ESP32** microcontroller with low-cost sensors. The mobile app configures the device's Wi-Fi network over Bluetooth Low Energy (the device advertises itself as `ESP32_Config`).

Component	Function
ESP32	Controller and Wi-Fi/BLE communication
DHT11	Air temperature and humidity
KY-038	Relative sound level
MQ-135	Gases and air quality; not a precision CO_2 meter and its voltage must be adapted before the ADC

Current hardware status. Test programs exist for the DHT11 and the KY-038. The final firmware that sends readings to the API, and the definitive MQ-135 wiring, are still pending confirmation. Therefore the system can receive measurements over HTTPS/MQTT, but the complete sensor → API chain is not considered closed in this deliverable.

9. Team and work organization

9.1 Team

Member	Role	Responsibility	Contact
María de los Ángeles Olaya García	Fullstack developer	Information analysis, design and programming	mariadelosangelesolayagar@gmail.com
Jhon Jairo Cortez Vargas	Fullstack developer and tech lead	Information analysis, design and programming; technical decisions	cortezvargasjhon01@gmail.com

9.2 Methodology

The team works with an agile approach based on **one-week sprints** and *spec-driven* documentation: the requirement, decision or contract is documented first, and then built. The practices applied in the repository are:

- **Documented governance:** agile conventions, definition of ready (DoR) and of done (DoD), Git conventions, documentation rules and security policies (00-governance folder).
- **Architecture decisions:** 17 numbered ADRs (ADR-001 to ADR-017) that record the reasoning behind each decision.
- **Branches by type of work:** `feat/`, `fix/`, `docs/` and `chore/`, merged through pull requests with a template.
- **Traceability:** user stories, a traceability matrix and functional and non-functional requirements linked to each other (04-requirements folder).
- **Contracts first:** each microservice publishes its OpenAPI contract before being integrated.

10. Current status and planning

10.1 Requirements status

According to SRS v1.1 (updated on 2026-10-08), the 40 functional requirements stand as follows:

Status	Count	Requirements
Implemented	26	RF1.1, 1.3–1.9; RF2.1, 2.4; RF3.1, 3.2; RF5.1; RF6.1, 6.2, 6.4; RF7.1–7.5; RF8.1; RF9.1–9.4
Partial	12	RF2.2, 2.3; RF3.3, 3.4; RF4.1, 4.2; RF5.2, 5.3; RF6.3; RF8.2, 8.3; RF9.5
Pending	2	RF1.2 (two-step verification); RF6.5 (external push or email notification)

10.2 Project milestones

Date	Milestone
2025	Initial version of the SRS (IEEE 830).
2026-09-03	Start of the engineering documentation repository (Docs-ea-control).
2026-09-15 / 17	Refinement of product, requirements and architecture; UX/UI documentation and web and mobile mockups.
2026-10-05	Analysis contract of ms-environment-monitoring.
2026-10-08	Source-code audit, risk register and SRS v1.1 aligned with the implementation.
2026-10-09	General document and consolidation of the project deliverables.

10.3 Roadmap

Horizon	Focus	Content	Exit condition
H1 – Now	Core MVP	Sign-up and authentication, environment and sensor management, measurement ingestion, dashboard and historical analysis, favorites and ratings.	A representative user completes the authorized consultation journey with valid data.
H2 – Next	Validated iteration	Improve dashboard clarity, filters, preferences and searches based on pilot evidence; strengthen data completeness and operational visibility.	Usage evidence identifies concrete improvements without leaving the product's domain.
H3 – Later	Controlled expansion	External notifications, richer operational analysis or other integrations, only with approved value, data ownership and contracts.	Each expansion has explicit requirements, data, API and architecture review.

11. Main risks

The risk register was updated from the code audit of 2026-10-08. The active risks are summarized below; their mitigation plans and the corrective, preventive and improvement actions are detailed in the **08_Calidad_Mejoras** document.

ID	Risk	Level
R-001	Identity spoofing through the X-User-* headers, which services accept without verifying their origin.	Critical
R-002	RSA private key committed in the ms-security repository.	Critical
R-003	Open CORS with credentials on the gateway.	High
R-004	Test coverage not enforced in continuous integration, despite NFR6.	High
R-005	ms-user-experience missing from the Kubernetes deployment and from continuous integration.	High
R-006	Observability documented but not effective in the code.	High
R-007	Gap between ADR-003 (database per service) and the actual deployment.	Medium
R-008	Circuit breaker documented (ADR-013) but not implemented.	Medium
R-009	Device firmware outside the API contract.	Medium
R-010	Frontend without tests or shared contracts.	Medium
R-011	Documentation describing elements that do not exist yet.	Medium

12. Project deliverables

The project is delivered in a folder with the following organization. This document is the first file; the others are built on the same basis and in the same format. Every document is delivered as PDF and Markdown, in Spanish and English.

Deliverable	Content
01_Documento_General_Proyecto	General project overview (this document).
02_Especificacion_Requisitos	SRS v1.1: functional and non-functional requirements, thresholds and use cases.
03_Analisis_Software	Software analysis and dynamic views.
04_Propuesta_Tecnica	Technical proposal, decisions and technology rationale.
05_Diseno_Software	Graphical interfaces, architectural model, design patterns and data model.
06_Base_Datos	Entity-relationship model, data dictionary and SQL script.
07_Pruebas	Test plan and evidence.
08_Calidad_Mejoras	Corrective, preventive and improvement actions.
09_Implementacion	Software implementation procedure.

Deliverable	Content
10_Manual_Tecnico · 11_Manual_Instalacion · 12_Manual_Usuario	Technical, installation and end-user manuals.
13_Articulo_Proyecto	Project article.
14_Anexos · 15_Enlace_Repositorio_GitHub	Supporting material and link to the code repository.

13. Glossary

Term	Description
Educational environment	Space (classroom, laboratory, library) monitored by EduAirControl.
Institution (tenant)	Educational organization using the platform; its data is isolated from that of others.
Campus	Subdivision of an institution that contains environments and sensors.
IoT	Internet of Things: network of connected sensors and microcontrollers.
ESP32	Microcontroller with Wi-Fi and Bluetooth used in the measurement devices.
BLE	Bluetooth Low Energy; used to configure the device from the mobile app.
OAuth2 / OIDC / PKCE	Authorization and identity standards used for login.
BFF	Backend for Frontend: the gateway stores the tokens and gives the browser only an httpOnly cookie.
RBAC	Role-based access control (USER, ADMIN, SUPER_ADMIN).
ADR	Architecture decision record.
SRS	Software Requirements Specification.

14. References

Organización Mundial de la Salud. (2021). *Guías sobre calidad del aire en interiores*. OMS.

Satish, U., Mendell, M. J., Shekhar, K., Hotchi, T., Sullivan, D., Streufert, S., & Fisk, W. J. (2012). Is CO_2 an indoor pollutant? Direct effects of low-to-moderate CO_2 concentrations on human decision-making performance. *Environmental Health Perspectives, 120*(12), 1671–1677.

U.S. Environmental Protection Agency. (2020). *Indoor air quality and student performance*. EPA.

Ministerio de Educación Nacional. (2022). *Lineamientos para ambientes escolares saludables*. MEN.

Naciones Unidas. (2015). *Objetivos de Desarrollo Sostenible – Agenda 2030*. ONU.

IEEE. (1998). *IEEE Std 830-1998: Recommended practice for software requirements specifications*. IEEE.

EduAirControl. (2026). *Software Requirements Specification, version 1.1*. SENA.

EduAirControl. (2026). *Docs-ea-control: ADR-001 – ADR-017, data model and OpenAPI contracts*. https://github.com/code-sena/ea-control-docs